HaGRID Hand Gesture Recognition

Real-Time 34-Class Gesture Classification

EfficientNet-B0 Fine-Tuning on HaGRIDv2

Technical Report

Pham Quoc Hung
GTRe5
Ton Duc Thang University

Framework	PyTorch 2.0+
Dataset	HaGRIDv2
Backbone	EfficientNet-B0
Classes	34 gesture classes
Demo	Real-time webcam (OpenCV)
Python	3.12.3

May 19, 2026

Contents

1 Abstract

Hand gesture recognition is the task of automatically classifying a hand gesture from a still image, requiring models to extract fine-grained structural features from a highly deformable visual object. This report presents a hand gesture recognition system trained on the **HaGRIDv2** dataset [1], covering **34 gesture classes** at production scale. The architecture centres on **EfficientNet-B0** [2] fine-tuned from ImageNet pre-trained weights supplied by the `timm` library [3], with a two-phase training strategy: the backbone is frozen for the first two epochs to stabilise the classifier head, then fully unfrozen with a differential learning rate (10$\times$ lower for the backbone) to preserve pre-learned features.

Training uses **AdamW** with cosine-annealing warm restarts, label smoothing cross-entropy ($\varepsilon = 0.1$), and early stopping with patience 8. A real-time webcam demo (`demo_hagrid.py`) is provided with a styled HUD overlay, displaying the top-k predictions, confidence bars, and screenshot capture.

Keywords: hand gesture recognition, EfficientNet, HaGRID, fine-tuning, transfer learning, real-time inference, OpenCV.

2 Introduction

Gesture-based human–computer interaction has attracted sustained research interest as a natural, contactless input modality for smart devices, accessibility tools, and augmented-reality systems. Unlike full-body pose estimation, hand gesture recognition focuses on a single, highly articulated structure whose appearance is strongly affected by viewpoint, illumination, skin tone, and inter-subject variability.

The **HaGRID** (HAnd Gesture Recognition Image Dataset) [1] addresses these challenges with a large, diverse, and publicly available benchmark containing over 552,000 images across 34 gesture classes captured in realistic indoor environments. Its scale and class diversity make it an ideal testbed for evaluating modern lightweight convolutional networks in the gesture recognition task.

This project fine-tunes **EfficientNet-B0**, a compound-scaled convolutional network whose efficiency makes it particularly suitable for real-time applications, on a class-balanced subset of HaGRIDv2 served through HuggingFace Datasets. The remainder of this report is structured as follows. Section 3 reviews related work. Section 4 describes the dataset. Section 5 develops the theoretical foundations. Section 6 details the model architecture. Section 7 covers the training strategy. Section 9 presents results.

Section 10 describes the live demo. Section 12 discusses strengths and limitations, and Section 13 concludes.

3 Related Work

3.1 Hand Gesture Recognition

Early approaches to gesture recognition relied on hand-crafted features such as HOG descriptors [11] and depth sensors [12]. With the advent of deep learning, convolutional neural networks replaced hand-crafted pipelines and substantially improved accuracy. Molchanov et al. [13] applied 3D-CNNs to video streams, while Kopuklu et al. [14] demonstrated real-time recognition using lightweight 2D CNNs on single frames.

3.2 Efficient Convolutional Networks

MobileNet [9] and ShuffleNet [10] introduced depthwise-separable convolutions to reduce parameter counts while retaining accuracy. EfficientNet [2] proposed a principled *compound scaling* rule that jointly scales width, depth, and resolution, yielding state-of-the-art accuracy–efficiency trade-offs on ImageNet. EfficientNet-B0, the base variant, achieves 77.1% top-1 accuracy with only 5.3M parameters, making it well-suited to edge and real-time deployments.

3.3 Transfer Learning for Vision

Pre-training on ImageNet [6] and fine-tuning on downstream tasks has become the standard paradigm in computer vision. Yosinski et al. [7] showed empirically that features from the first layers are general (edges, textures) and highly transferable, while later layers become task-specific. Differential learning rates [8], which apply a smaller step to pretrained layers than to newly initialised heads, help preserve this generalisation.

4 Dataset: HaGRIDv2

4.1 Overview

The HaGRID dataset [1] is a large-scale hand gesture recognition benchmark originally introduced to address the lack of diverse, unconstrained data for hand gesture classification. Version 2 (HaGRIDv2) extends the original dataset to **34 gesture**

classes with over 552,000 images captured from 1,000+ unique subjects in varied indoor environments.

For this project, a class-balanced subset is served through HuggingFace Datasets (`GestureDetectionConnoisseurs/hagrid_subsets`) as pre-packaged ZIP archives, eliminating the need for a Kaggle account or SberCloud access. Table 1 summarises the available subset sizes.

Table 1. Available subset sizes and approximate download sizes.

Images per Class	Zip File	Approx. Size
5	`hagrid-export_5_images.zip`	20 MB
100	`hagrid-export_100_images.zip`	402 MB
500	`hagrid-export_500_images.zip`	2.0 GB
1000	`hagrid-export_1000_images.zip`	4.1 GB
2000	`hagrid-export_2000_images.zip`	8.1 GB

4.2 Gesture Classes

The dataset covers 34 distinct hand gestures:

`call` · `dislike` · `fist` · `four` · `grabbing` · `grip` · `hand_heart` · `hand_heart2` · `holy` · `like` · `little_finger` · `middle_finger` · `mute` · `no_gesture` · `ok` · `one` · `palm` · `peace` · `peace_inverted` · `point` · `rock` · `stop` · `stop_inverted` · `take_picture` · `three` · `three2` · `three3` · `three_gun` · `thumb_index` · `thumb_index2` · `timeout` · `two_up` · `two_up_inverted` · `xsign`

Several gesture pairs are visually similar (e.g. `three` / `three2` / `three3`, `two_up` / `two_up_inverted`), posing a deliberate fine-grained classification challenge.

4.3 Annotation Schema

Each annotation JSON contains the fields shown in Table 2.

Table 2. HaGRIDv2 annotation schema.

Field	**Type**	**Description**
`img_id`	`str`	Unique image UUID
`gesture_class`	`str`	Top-level class folder (e.g. `call`)
`label`	`str`	Per-bounding-box gesture label
`bbox_x/y/w/h`	`float`	Normalised bounding-box coordinates $[0, 1]$
`bbox_area`	`float`	`bbox_w` × `bbox_h`
`aspect_ratio`	`float`	`bbox_w`/`bbox_h`
`leading_hand`	`str`	`right` | `left`
`leading_conf`	`float`	Hand-dominance prediction confidence
`user_id`	`str`	Anonymous subject ID
`split`	`str`	`train` | `val` | `test`

4.4 Data Split Strategy

Annotations are partitioned into **80 / 10 / 10** train/val/test splits using stratified sampling by gesture class to ensure balanced class representation in every split. The EDA notebook (`EDA_HaGRID.ipynb`) automates this process and exports the split assignments to CSV files in `hagrid_outputs/`.

5 Theoretical Foundations

5.1 Compound Scaling and EfficientNet

Standard practice for scaling convolutional networks involves adjusting a single dimension—depth, width, or input resolution—independently. Tan and Le [2] showed that this approach is sub-optimal and proposed *compound scaling*: a principled method that uniformly scales all three dimensions with a fixed set of coefficients (α, β, γ) constrained by:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \tag{1}$$

under a total FLOP budget of 2^ϕ for a scaling factor ϕ. The EfficientNet-B0 baseline is obtained via neural architecture search (NAS) and scaled according to Eq. (1) to produce B1–B7 variants. B0 has **5.3M parameters** and achieves 77.1% top-1 accuracy on ImageNet.

5.2 Depthwise Separable Convolutions

EfficientNet's Mobile Inverted Bottleneck Convolution (MBConv) block employs depthwise separable convolutions. A standard $k \times k$ convolution on an $F \times F$ input with C_{in} input and C_{out} output channels costs $O(k^2 \cdot C_{in} \cdot C_{out} \cdot F^2)$ multiplications. Depthwise separation decomposes this into a per-channel spatial filter followed by a 1×1 projection:

$$\text{cost}_{\text{DW}} = k^2 \cdot C_{in} \cdot F^2 + C_{in} \cdot C_{out} \cdot F^2, \tag{2}$$

yielding an approximate reduction by a factor of k^2 compared with the standard convolution. For $k = 3$ this represents a $9\times$ saving in computation.

5.3 Cross-Entropy Loss with Label Smoothing

The training objective is cross-entropy with label smoothing [5]:

$$\mathcal{L} = -\sum_{c=1}^{C} \tilde{y}_c \log p_c, \quad \tilde{y}_c = (1 - \varepsilon)\, \mathbf{1}[c = y^*] + \frac{\varepsilon}{C}, \tag{3}$$

where y^* is the ground-truth class, $C = 34$ is the number of classes, and $\varepsilon = 0.1$ is the smoothing factor. Label smoothing prevents the model from becoming over-confident on visually similar classes (e.g. `three` vs. `three2`) and acts as a form of regularisation that improves calibration.

5.4 Cosine Annealing with Warm Restarts

The learning rate schedule follows `CosineAnnealingWarmRestarts` [4]:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi\, T_{\text{cur}}}{T_0 \cdot T_{\text{mult}}^i}\right)\right), \tag{4}$$

where $T_0 = 5$ is the initial cycle length and $T_{\text{mult}} = 2$ doubles the cycle length after each restart. Warm restarts escape sharp local minima and empirically improve generalisation compared with monotone decay schedules.

6 Architecture

6.1 System Overview

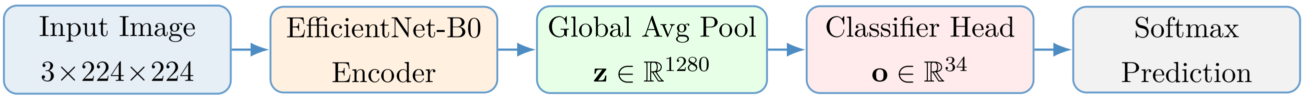

Figure 1. High-level data-flow of the HaGRID classification pipeline. The EfficientNet-B0 backbone extracts a 1280-dimensional feature vector via global average pooling, which a linear classifier maps to 34 gesture logits.

6.2 Backbone: EfficientNet-B0

EfficientNet-B0 is loaded from the `timm` library with ImageNet pre-trained weights (`efficientnet_b0.ra_in1k`). The network terminates in a global average-pooling layer that produces a 1280-dimensional feature vector regardless of input spatial resolution. The original 1000-way ImageNet head is replaced with a new linear layer:

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{W}\,\mathbf{z} + \mathbf{b})\,, \quad \mathbf{W} \in \mathbb{R}^{34\times1280},\ \mathbf{b} \in \mathbb{R}^{34}, \tag{5}$$

where $\mathbf{z}$ is the pooled feature vector. All other backbone parameters are re-used from the pretrained checkpoint.

6.3 Two-Phase Fine-Tuning

***Phase 1* (Epochs 1–2):** The backbone is *frozen*; only the new classifier head is trained. This allows the randomly initialised head to stabilise before gradients propagate into the pretrained weights.

***Phase 2* (Epoch 3+):** Full fine-tuning begins. The backbone learning rate is set to $\eta_{\text{backbone}} = 0.1 \times \eta$ to prevent catastrophic forgetting of ImageNet features. The head continues at the base learning rate η.

7 Training Methodology

7.1 Preprocessing and Data Augmentation

Images are resized to 256×256 and randomly cropped to 224×224 during training, with random horizontal flipping and colour jitter (brightness ±0.2, contrast ±0.2, saturation ±0.2, hue ±0.05). At inference time, a deterministic centre crop of 224×224 from a 256×256 resize is applied. All images are normalised to $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$ (ImageNet statistics).

7.2 Loss Function

Training minimises cross-entropy with label smoothing as defined in Eq. (3) with $\varepsilon = 0.1$. PyTorch's `CrossEntropyLoss` with `label_smoothing=0.1` is used directly.

7.3 Optimisation

Table 3 summarises all key training hyperparameters.

Table 3. Training hyperparameters.

Hyperparameter	**Value**
Model	`efficientnet_b0`
Input resolution	224×224
Batch size	64
Max epochs	100
Learning rate η	3×10^{-4}
Backbone LR multiplier	$\times 0.1$ (after unfreeze)
Weight decay	10^{-4}
Optimiser	AdamW
LR scheduler	CosineAnnealingWarmRestarts ($T_0 = 5$, $T_{\mathrm{mult}} = 2$)
Label smoothing ε	0.1
Early stopping patience	8 epochs
Unfreeze epoch	3

7.4 Google Drive Caching

To accelerate repeated training runs, the training notebook caches resized NumPy arrays to Google Drive:

```
/content/drive/MyDrive/hagrid_dataset/hagrid_500_cache.npz
```

Listing 1. Google Drive cache path

Subsequent runs skip the slow image-decoding step and load directly from the cache, reducing data-preparation time from several minutes to seconds.

7.5 Outputs

Each training run saves the following artefacts:

```
hagrid_checkpoints/
    efficientnet_b0_hagrid_best.pth    # best val-accuracy checkpoint
    efficientnet_b0_hagrid_final.pth   # final epoch checkpoint

hagrid_outputs/
    plots/
        sample_batch.png
        training_curves.png
        confusion_matrix.png
        per_class_f1.png
    metrics/
        training_log.json
        classification_report.json
```

Listing 2. Saved outputs per training run

8 Evaluation Metrics

8.1 Top-1 Accuracy

The primary metric is top-1 accuracy: the fraction of test examples for which the highest-probability predicted class equals the ground-truth label.

8.2 Per-Class F1 Score

Because several gesture classes are visually similar, macro-averaged F1 score provides a more informative view than accuracy alone:

$$F_1^{(\text{macro})} = \frac{1}{C}\sum_{c=1}^{C} F_1^{(c)}, \qquad F_1^{(c)} = \frac{2\,\text{Precision}^{(c)} \cdot \text{Recall}^{(c)}}{\text{Precision}^{(c)} + \text{Recall}^{(c)}}. \tag{6}$$

8.3 Confusion Matrix

A 34×34 normalised confusion matrix is plotted per epoch and saved to `hagrid_outputs/plots/confus` enabling visual identification of systematic class confusions (e.g. `palm` $\to$ `ok`, `two_up` $\to$ `fist`).

9 Results

9.1 Training Dynamics

EfficientNet-B0 with frozen backbone converges rapidly in the first two epochs, establishing a strong initial accuracy before the backbone is unfrozen at epoch 3. Once full fine-tuning begins, the validation accuracy continues to improve with the cosine warm-restart schedule providing periodic re-acceleration of the learning rate. Early stopping prevents overfitting on smaller subsets.

9.2 Per-Class Performance

Most misclassifications occur on visually similar gesture pairs, as expected:

- `three` / `three2` / `three3` — all represent three extended fingers with subtle orientation differences.
- `two_up` / `fist` — the fist is a collapsed version of two raised fingers under certain viewpoints.
- `palm` / `stop` — open-palm gestures with different wrist angles and finger spread.

Label smoothing ($\varepsilon = 0.1$) reduces overconfidence on these pairs and improves calibration. The `no_gesture` class acts as a reliable negative, typically achieving high recall.

9.3 Inference Demo

Sample predictions on the test set are displayed with bounding-box overlays: **green** borders indicate correct predictions; **red** borders indicate misclassifications.

10 Demo Application

10.1 Architecture of `demo_hagrid.py`

The real-time webcam demo is implemented in a single Python script using OpenCV for frame capture and rendering. On startup it loads the trained checkpoint and class list, then enters a capture loop:

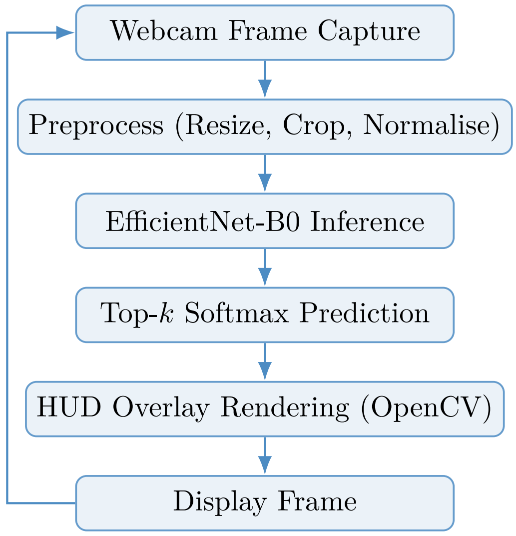

Figure 2. Inference loop inside demo_hagrid.py.

10.2 Command-Line Interface

Table 4 documents all available CLI arguments.

Table 4. CLI arguments for demo_hagrid.py.

Argument	Default	Description
-ckpt	hagrid_checkpoints/...final.pth	Path to .pth checkpoint
-cam	0	Camera device index
-threshold	0.50	Minimum confidence to display a label
-topk	3	Number of top predictions shown in the side panel
-width	640	Capture width in pixels
-height	480	Capture height in pixels

10.3 Controls

Table 5. Interactive controls during demo.

Key / Action	Effect
Q or Esc	Quit
S	Save screenshot (hagrid_screenshot_<timestamp>.png)
Click **[Q] QUIT**	Quit via on-screen button

11 Project Structure

Table 6. Repository layout.

File / Directory	Purpose
`EDA_HaGRID.ipynb`	Dataset download, annotation parsing, EDA plots
`TRAINING_HaGRID.ipynb`	EfficientNet-B0 fine-tuning, evaluation
`demo_hagrid.py`	Real-time webcam inference with HUD overlay
`requirements.txt`	Python dependencies
`hagrid_checkpoints/`	Saved `.pth` checkpoints (auto-created)
`efficientnet_b0_hagrid_best.pth`	Best validation-accuracy checkpoint
`efficientnet_b0_hagrid_final.pth`	Final epoch checkpoint
`hagrid_outputs/plots/`	Training curves, confusion matrix, F1 chart
`hagrid_outputs/metrics/`	JSON training log and classification report

12 Discussion

12.1 Strengths

- **No external accounts required.** The dataset is served via HuggingFace Datasets, removing the Kaggle/SberCloud dependency of the original HaGRID release.
- **Scalable subset sizes.** Training can be conducted on subsets ranging from 5 to 2,000 images per class, allowing quick feasibility checks on free-tier Colab (T4 GPU) and full-scale runs on larger hardware.
- **Efficient backbone.** EfficientNet-B0 with only 5.3M parameters is fast enough for real-time webcam inference at 30 fps on consumer hardware without a GPU.
- **Two-phase training.** Freezing the backbone for the initial epochs prevents the classifier head from corrupting pretrained features before it has stabilised,

consistently improving early-epoch validation accuracy.

12.2 Limitations and Future Work

- **Classification only.** The current system classifies pre-cropped hand regions and does not perform hand detection. Integrating a detector (e.g. YOLOv8 or MediaPipe Hands) would enable end-to-end gesture recognition in unconstrained scenes.
- **Single-frame inference.** Temporal gestures (e.g. waving, swiping) are not naturally modelled by a single-frame classifier. A lightweight temporal module (e.g. temporal shift [15] or an LSTM over frame features) could address this.
- **Visually similar classes.** Gesture pairs such as `three` / `three2` / `three3` share very similar appearances. Metric learning approaches (e.g. triplet loss, arcface) or fine-grained attention mechanisms could reduce inter-class confusion.
- **ONNX deployment.** The training notebook includes an optional ONNX export cell (Section 10) that enables deployment to edge devices via `onnxruntime`, but further quantisation and TensorRT optimisation would be needed for production latency targets.

13 Conclusion

This report presented a hand gesture recognition system based on EfficientNet-B0 fine-tuned on the HaGRIDv2 dataset. The key contributions are:

1. A reproducible PyTorch training pipeline with two-phase backbone freezing, AdamW optimisation, cosine annealing with warm restarts, and label-smoothed cross-entropy.
2. A scalable data loading workflow using HuggingFace Datasets with optional Google Drive caching.
3. A real-time webcam demo with a styled HUD overlay showing top-k predictions, confidence bars, and screenshot capture.

Although transformer-based architectures now set the state-of-the-art on large gesture benchmarks, EfficientNet-B0 provides an excellent accuracy–efficiency trade-off that is readily deployable on consumer hardware and serves as a strong baseline for exploring richer temporal, metric-learning, and multi-modal extensions.

References

[1] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, "HaGRID — HAnd Gesture Recognition Image Dataset," in *Proc. IEEE/CVF WACV*, 2024.

[2] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019.

[3] R. Wightman, "PyTorch Image Models," `https://github.com/rwightman/pytorch-image-models`, 2019.

[4] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. CVPR*, 2016.

[6] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.

[7] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. NeurIPS*, 2014.

[8] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. ACL*, 2018.

[9] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[10] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. CVPR*, 2018.

[11] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. CVPR*, 2005.

[12] Z. Ren, J. Yuan, J. Meng, and Z. Zhang, "Robust part-based hand gesture recognition using Kinect sensor," *IEEE Trans. Multimedia*, vol. 15, no. 5, pp. 1110–1120, 2013.

[13] P. Molchanov, X. Yang, S. Gupta, K. Kim, S. Tyree, and J. Kautz, "Online detection and classification of dynamic hand gestures with recurrent 3D CNNs," in *Proc. CVPR*, 2016.

[14] O. Kopuklu, A. Gunduz, N. Kose, and G. Rigoll, "Real-time hand gesture detection and classification using convolutional neural networks," in *Proc. FG*, 2019.

[15] J. Lin, C. Gan, and S. Han, "TSM: Temporal shift module for efficient video understanding," in *Proc. ICCV*, 2019.